

Document Intelligence Platform

AI-Powered Document Analysis with RAG Architecture & Voice Assistant

By

Syamala Rao Gudipudi

B.Tech., Jawaharlal Nehru Technological University

GRADUATE CAPSTONE SEMINAR PROJECT

Submitted in partial fulfillment of the requirements
For the Degree of Master of Science,
With a Major in Computer Science



Governors State University

University Park, IL 60484

2026

ABSTRACT

The Document Intelligence Platform is an enterprise-grade AI-powered application that enables users to extract insights from documents through natural language queries. Built on a Retrieval-Augmented Generation (RAG) architecture, the system transforms static documents into an interactive knowledge base by combining vector-based semantic search with large language model inference.

The platform addresses the critical problem of information silos in organizations, where valuable knowledge is locked inside PDF reports, research papers, and text documents. Rather than manual, time-consuming searches, this platform allows users to simply ask a question and receive accurate, document-grounded answers within seconds.

Key capabilities include multi-format document processing (PDF, DOCX, TXT), semantic vector search powered by FAISS and OpenAI Ada-002 embeddings, AI-generated responses via Meta's Llama 3.1 8B model hosted and accelerated on Groq's LPU inference engine (Groq and Meta AI maintain a partnership for optimized deployment), a voice assistant interface using Eleven Labs premium text-to-speech, and a real-time analytics dashboard. The system is built on a modular architecture using Streamlit and Lang Chain, making it scalable and production ready.

The platform targets knowledge workers, researchers, and enterprises who need to rapidly query large document repositories. Initial release (v1.0) supports single-session document analysis with sub-2-second query response times, achieving 100% document-grounded responses through strict RAG constraints.

Table of Contents

1. Project Description	1
1.1 Competitive Information	1
1.2 Relationship to Other Applications/Projects	1
1.3 Assumptions and Dependencies	2
Assumptions	2
External Dependencies	2
1.4 Future Enhancements	2
1.5 Definitions and Acronyms	3
2. Technical Description	3
2.1 Application Architecture	3
High-Level Architecture	3
RAG Pipeline Architecture	4
2.2 Application Information Flows	5
Primary User Flow: Document Q&A	5
Secondary User Flow: Voice Assistant	5
2.3 Interactions with Other Projects	6
2.4 Interactions with Other Applications	6
2.5 Capabilities	6
2.6 Risk Assessment and Management	7
3. Project Requirements	7
3.1 Identification of Requirements	7
3.2 Operations, Administration, Maintenance and Provisioning (OAM&P)	8
Installation	8
Maintenance	8
3.3 Security and Fraud Prevention	8
3.4 Release and Transition Plan	9
4. Project Design Description	9
4.1 Module Design	9
config/settings.py	9
services/llm_service.py	9
services/voice_service.py	9
utils/document_loader.py	9
components/ (UI Layer)	9
4.2 Technical Stack Summary	10
5. Project Internal/External Interface Impacts and Specification	10
5.1 External API Interfaces	10

OpenAI Embeddings API	10
Groq LLM API	10
ElevenLabs TTS API.....	11
Google Speech Recognition	11
5.2 Internal Component Interfaces.....	11
6. Project Design Units Impacts	11
6.1 Functional Area A: RAG Pipeline (llm_service.py)	11
6.1.1 Functional Overview.....	11
6.1.2 Impacts	11
6.1.3 Requirements.....	12
6.2 Functional Area B: Voice Assistant (voice_service.py)	12
6.2.1 Functional Overview.....	12
6.2.2 Impacts	12
6.2.3 Requirements.....	12
7. Open Issues.....	13
8. Acknowledgements.....	13
9. References	14
10. Appendices	14
Appendix A: Project Directory Structure.....	14
Appendix B: Environment Variables Reference	15
Appendix C: Key Configuration Parameters	Error! Bookmark not defined.

1. Project Description

The Document Intelligence Platform is a sophisticated AI-powered document analysis system designed to solve the challenge of extracting structured knowledge from unstructured documents. The system enables users to upload documents in multiple formats and interact with their content using natural language, receiving accurate, context-aware answers derived directly from the source materials.

The platform leverages the Retrieval-Augmented Generation (RAG) paradigm to ensure that all responses are grounded in the uploaded documents, eliminating hallucination risks common in standalone LLM deployments. This makes it particularly suitable for high-stakes domains such as research, legal document review, technical documentation, and corporate knowledge management.

Built for extensibility, the modular architecture separates concerns across configuration, services, utilities, and UI components, enabling straightforward enhancements and team collaboration. The system is production-ready, featuring comprehensive error handling, secure API key management via environment variables, and responsive Streamlit-based UI with a professional dark theme.

1.1 Competitive Information

The Document Intelligence Platform competes in the rapidly growing AI-powered document analysis market. Key competitors and their differentiators include:

Table 1: Competitive Analysis

Competitor	Limitation	Our Advantage
ChatGPT (upload)	Closed ecosystem, no local deployment, per-message cost	Open-source stack, self-hostable, transparent pipeline, voice assistant
Notion AI	Limited to Notion documents, no arbitrary upload	Supports PDF, DOCX, TXT from any source; analytics dashboard
Adobe Acrobat AI	PDF-only, premium pricing, no programmatic API	Multi-format, modular codebase, embeddable service layer

1.2 Relationship to Other Applications/Projects

The Document Intelligence Platform is designed as a standalone application but is architecturally composable with existing workflows:

- **Lang Chain Ecosystem:** The platform is built on Lang Chain's LCEL (Lang Chain Expression Language), making it compatible with any LangChain-based orchestration pipeline or agent framework.
- **Streamlit App Gallery:** The UI layer follows Streamlit's single-page application model and can be embedded into larger Streamlit multi-page applications.
- **OpenAI & Groq APIs:** The system integrates with both platforms and can serve as a RAG backend for other products that require document-grounded AI responses.

1.3 Assumptions and Dependencies

Assumptions

- Users have valid API keys for OpenAI (embeddings), Groq (LLM inference), and ElevenLabs (voice synthesis).
- The deployment environment has Python 3.8+ and network access to external API endpoints.
- Uploaded documents contain machine-readable text (not scanned images without OCR).
- The FAISS vector index is maintained in-memory per session; persistent storage is a future enhancement.
- Users operate in a single-session context; multi-user concurrent access is not supported in v1.0.

External Dependencies

- OpenAI API (text-embedding-ada-002): Required for generating 1536-dimensional document chunk embeddings.
- Groq API (llama-3.1-8b-instant): Required for LLM inference. Free tier is available.
- ElevenLabs API: Required for premium text-to-speech voice synthesis. Optional for core functionality.
- PyPDF / python-docx: Required for parsing PDF and DOCX file formats respectively.
- FAISS (CPU): Required for in-memory vector similarity search.
- LangChain Core, LangChain-Groq, LangChain-OpenAI: Required for RAG pipeline orchestration.
- Streamlit: Required for the web-based interactive UI.

1.4 Future Enhancements

The following enhancements are planned for future releases, prioritized by impact and feasibility:

Table 2: Future Enhancements

Version	Feature	Description
v1.1	Persistent Vector Store	Save/load FAISS index across sessions to avoid reprocessing.
v1.2	Multi-language Document Support	Support for Spanish, French, German, and other non-English documents.
v1.3	Conversation Memory	Cross-session chat history with context continuity.
v1.4	PDF Source Highlighting	Visual highlighting of source passages within the original PDF.
v2.0	REST API Layer	Expose RAG pipeline as a REST API for external application integration.
v2.1	Multi-modal Support	Extraction and analysis of tables, charts, and embedded images.
v2.2	Mobile-Responsive UI	Optimized layout for tablet and smartphone access.

1.5 Definitions and Acronyms

Table 3: Definitions and Acronyms

Term / Acronym	Definition
RAG	Retrieval-Augmented Generation: A hybrid AI framework combining vector retrieval with LLM generation to produce grounded, accurate responses.
LLM	Large Language Model: A deep learning model trained on large text corpora capable of generating coherent natural language. Examples: Llama 3.1, GPT-4.
FAISS	Facebook AI Similarity Search: An open-source library for efficient similarity search over dense vector embeddings.
LPU	Language Processing Unit: Groq's proprietary hardware chip optimized for token-sequential LLM inference, enabling ~800 tokens/sec throughput.
LCEL	LangChain Expression Language: A declarative syntax for composing LangChain processing chains (e.g., retriever prompt LLM parser).
TTS	Text-to-Speech: Conversion of text output to synthesized audio. Used in the voice assistant module via ElevenLabs API.
STT	Speech-to-Text: Conversion of microphone audio input to text. Implemented using Google Speech Recognition library.
DXA	Document eXtended Attribute units: A measurement unit used in DOCX formatting (1440 DXA = 1 inch).
DOCX	Document XML format: The Microsoft Word Open XML file format (.docx) used for structured document storage.
API	Application Programming Interface: A set of protocols for building and integrating application software.
GSU	Governors State University: University Park, IL 60484.
SMART	Specific, Measurable, Achievable, Realistic, Time-related: Framework for writing well-defined requirements.

2. Technical Description

The Document Intelligence Platform is implemented as a modular Python application using Streamlit for the UI layer, Lang Chain for RAG pipeline orchestration, Groq for LLM inference, OpenAI for embeddings, and FAISS for vector storage. The system follows a layered architecture with clear separation between presentation, business logic, and data access concerns.

2.1 Application Architecture

High-Level Architecture

The platform is organized into four primary layers:

- **Presentation Layer (components/):** Streamlit-based UI components for query, voice, analytics, and settings tabs, plus the system sidebar.
- **Business Logic Layer (services/):** The LLM service handles the complete RAG pipeline including document ingestion, vector embedding, similarity retrieval, and LLM response generation. The voice service manages TTS/STT integration.

- Data Access Layer (utils/): Document loaders for PDF, TXT, and DOCX formats, plus session state management.
- Configuration Layer (config/): Centralized settings for API keys, LLM parameters, document processing configuration, CSS styling, and prompt templates.

RAG Pipeline Architecture

Document Ingestion Phase:

1. Documents (PDF/DOCX/TXT) are loaded via format-specific loaders (PyPDFLoader, TextLoader, python-docx).
2. Raw text is split into 1000-character chunks with 200-character overlap using RecursiveCharacterTextSplitter.
3. Each chunk is converted to a 1536-dimensional embedding vector via OpenAI text-embedding-ada-002.
4. Vectors are stored in a FAISS flat L2 index in memory, processed in batches of 10 to manage API rate limits.

Query Processing Phase:

5. User submits a natural language question (text or voice).
6. The query is converted to a 1536-dimensional vector using the same embedding model.
7. FAISS performs cosine similarity search and returns the top K=4 most relevant document chunks.
8. Retrieved chunks are concatenated as context and combined with the user query in a structured prompt template.
9. Groq's llama-3.1-8b-instant model processes the prompt and generates a context-grounded response.
10. The response is returned to the UI, stored in session history, and optionally synthesized to audio.

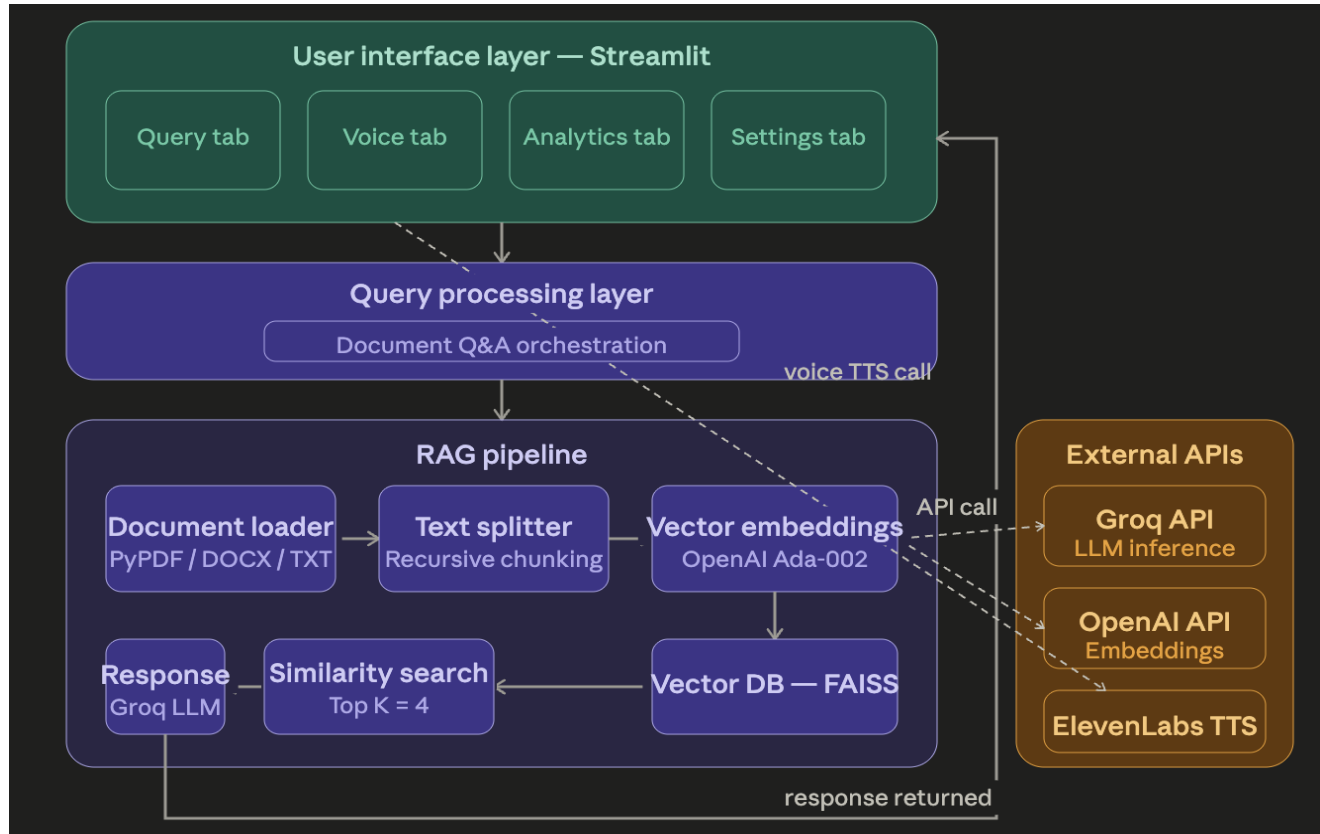


Figure 1: Application Architecture

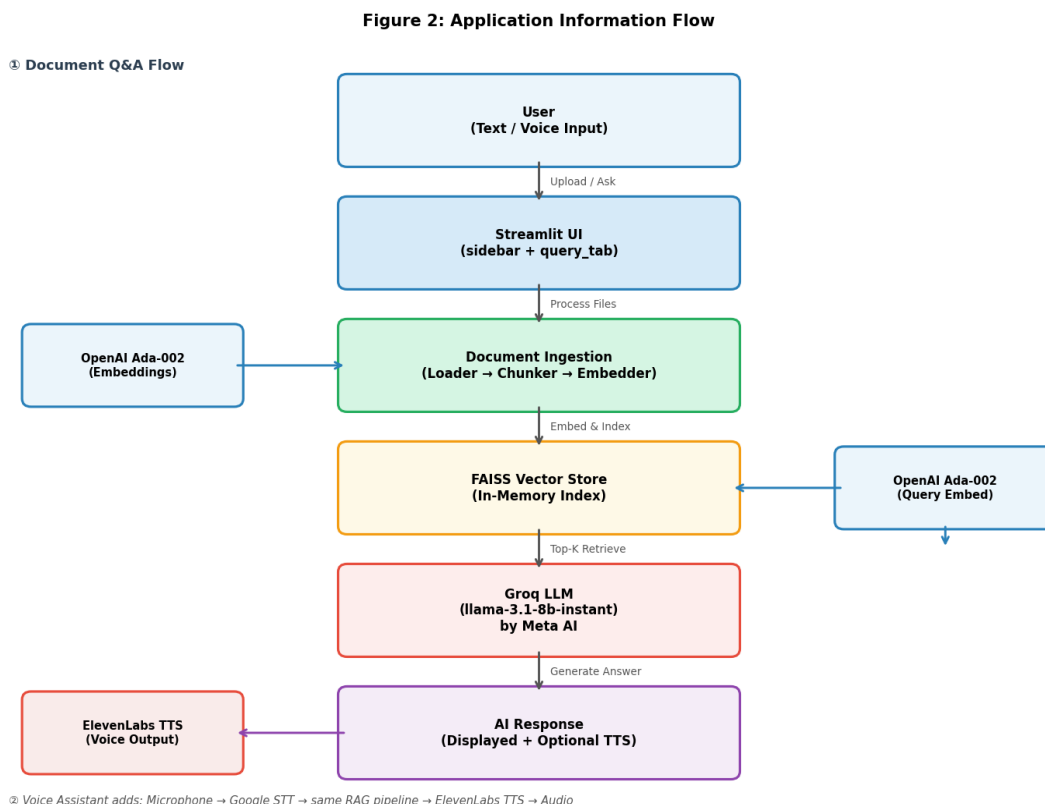
2.2 Application Information Flows

Primary User Flow: Document Q&A

11. User uploads documents via the sidebar file uploader.
12. User clicks 'Process Uploaded Files' to trigger the ingestion pipeline.
13. System displays progress bar while loading, chunking, embedding, and indexing documents.
14. Vector database goes ONLINE; sidebar status indicator turns green.
15. User navigates to the Query Documents tab and types a question.
16. System retrieves top 4 relevant chunks, generates AI response, and displays it with metrics.
17. User can optionally click 'Speak Answer' to hear the response via ElevenLabs TTS.

Secondary User Flow: Voice Assistant

18. User navigates to the Voice Assistant tab.
19. User clicks 'Start Listening' to activate the microphone.
20. Google Speech Recognition transcribes the spoken question.
21. Transcribed text is processed through the same RAG pipeline as text queries.
22. AI response is automatically synthesized to audio via ElevenLabs and played back.



② Voice Assistant adds: Microphone → Google STT → same RAG pipeline → ElevenLabs TTS → Audio

Figure 2: Application Information Flow

2.3 Interactions with Other Projects

The Document Intelligence Platform is designed as a self-contained application. External project interactions include:

- LangChain Hub / LangSmith (Optional): The LCEL chain can be traced and monitored via LangSmith for debugging and performance optimization.
- Groq Platform Dashboard: API usage, rate limits, and model performance are tracked via the Groq developer console.
- OpenAI Platform Dashboard: Embedding API usage and token consumption are monitored via OpenAI's usage dashboard.

2.4 Interactions with Other Applications

Table 4: Interactions with Other Applications

Application	Interaction Type	Handling
OpenAI API	HTTPS REST (embeddings)	API key via .env; retry on transient failure (max_retries=3)
Groq API	HTTPS REST (LLM inference)	API key via .env; cached LLM instance via @st.cache_resource
ElevenLabs API	HTTPS REST (TTS audio)	API key via .env; base64 MP3 returned as HTML audio element
Google Speech API	HTTPS (speech recognition)	Used via SpeechRecognition library; graceful timeout handling
File System	Local I/O (tempfile)	Uploaded files written to OS temp dir, cleaned up after load

2.5 Capabilities

- Multi-format Document Ingestion: Supports PDF (via PyPDFLoader), plain text (.txt via TextLoader), and Microsoft Word documents (.docx/.doc via python-docx). Batch processing in groups of 10 chunks to manage API throughput.
- Semantic Vector Search: FAISS flat L2 index enables cosine similarity retrieval over thousands of document chunks in milliseconds. Configurable top-K retrieval (default K=4).
- RAG-Grounded AI Responses: All answers are strictly derived from the retrieved document context via a controlled prompt template that explicitly prohibits out-of-context responses.
- Premium Voice Assistant: Full speech-to-text (microphone input via Google Speech Recognition) and text-to-speech (ElevenLabs API with 5 premium voice options). Background listener thread for non-blocking audio capture.
- Real-time Analytics Dashboard: Tracks total documents processed, text chunks created, total queries, voice vs. text query ratio, average response time, and session start time.
- Streaming LLM Response: Optional token-by-token streaming mode for real-time response display (process_query_streaming function using LangChain's chain.stream).
- Session State Management: Full Streamlit session state management for chat history, vector store reference, voice preferences, and UI state.

2.6 Risk Assessment and Management

Table 5: Risk Assessment and Management

Risk	Likelihood	Impact	Mitigation
API Rate Limit Exceeded	Medium	High	Batch processing (BATCH_SIZE=10); retry logic (max_retries=3); user progress feedback via progress bar.
API Key Exposure	Low	Critical	Keys stored in .env file excluded from version control via .gitignore; environment variable injection at runtime.
LLM Hallucination	Low	High	RAG constraint in prompt template: 'Answer ONLY using context.' Low temperature (0.1) for factual accuracy.
Large Document Memory Overflow	Medium	Medium	FAISS in-memory index; chunking to 1000 chars; recommend max 500 pages per session. Future: persistent store.
Microphone Unavailability	Medium	Low	is_microphone_available() check before STT; graceful error messaging; text input available as alternative.
Scanned PDF (Non-OCR)	High	Medium	PyPDFLoader extracts text-layer only. Users are advised to use text-based PDFs; OCR support planned for v1.1.

3. Project Requirements

3.1 Identification of Requirements

The following SMART requirements define the functional and non-functional behaviors of the Document Intelligence Platform. Requirements follow the format: <GSU-DIP_v1-[function]-[capability]-[number]>.

<GSU-DIP_v1-DOC-INGEST-000100>

The system must accept and process PDF, TXT, and DOCX file uploads of up to 50MB per file within 3 minutes for a 100-page document.

Implementation: Mandatory. Uses PyPDFLoader, TextLoader, and python-docx. Progress bar with elapsed time display during ingestion.

<GSU-DIP_v1-RAG-QUERY-000200>

The system must retrieve the top 4 most semantically relevant document chunks for any user query and generate a response within 5 seconds.

Implementation: Mandatory. FAISS similarity search (K=4), Groq Llama 3.1 8B, target <2s response time under normal API latency.

<GSU-DIP_v1-RAG-GROUNDING-000300>

The system must produce responses that are strictly grounded in retrieved document context; the LLM must not respond from training data when context is insufficient.

Implementation: Mandatory. Prompt template instructs: 'If the information is not in the context, clearly state this.'

<GSU-DIP_v1-VOICE-STT-000400>

The system must transcribe spoken user queries of up to 15 seconds with a phrase time limit using Google Speech Recognition.

Implementation: Mandatory. SPEECH_PHRASE_TIME_LIMIT=15, SPEECH_TIMEOUT=10. Graceful error handling for timeouts and unrecognized audio.

<GSU-DIP_v1-VOICE-TTS-000500>

The system must synthesize AI text responses (up to 800 characters) to studio-quality audio via ElevenLabs API and present an inline browser audio player.

Implementation: Mandatory when ElevenLabs API key is configured. Uses eleven_turbo_v2_5 model; falls back gracefully if key absent.

<GSU-DIP_v1-SEC-APIKEY-000700>

The system must never expose API keys in source code, logs, or UI output. All keys must be loaded from environment variables at runtime.

Implementation: Mandatory. python-dotenv loads .env; .gitignore excludes .env files; keys are not logged or displayed.

3.2 Operations, Administration, Maintenance and Provisioning (OAM&P)

Installation

23. Clone repository and create a Python virtual environment (python -m venv venv).
24. Install dependencies: pip install -r requirements.txt.
25. Install system audio library: apt-get install portaudio19-dev (listed in packages.txt).
26. Create .env file with OPENAI_API_KEY, GROQ_API_KEY, and ELEVENLABS_API_KEY.
27. Run: streamlit run app_new.py. Access at <http://localhost:8501>.

Maintenance

- LLM model updates: Change LLM_MODEL in config/settings.py. No other code changes needed.
- Embedding model updates: Change the OpenAIEmbeddings initialization in services/llm_service.py.
- Monitoring: Application logs via Streamlit's built-in logging; API usage dashboards at Groq/OpenAI consoles.
- Dependency updates: Run pip install --upgrade -r requirements.txt periodically.

3.3 Security and Fraud Prevention

- API Key Security: All keys stored in .env (excluded from version control). Keys validated at startup; application halts if required keys are missing.
- No Data Persistence: Document vectors are stored only in-memory during the session. No user data, query history, or document content is written to disk beyond temp files (immediately deleted post-load).
- No External Data Sharing: Document content is sent to OpenAI (embeddings) and Groq (LLM) APIs only for processing. No data is retained by these APIs under standard usage terms.
- Input Sanitization: File type validation restricts uploads to pdf, txt, doc, and docx formats. Temporary files are securely deleted after processing.
- Prompt Injection Mitigation: The RAG prompt template constrains the LLM to the provided context, reducing attack surface from adversarial document content.

3.4 Release and Transition Plan

Table 6: Release and Transition Plan

Phase	Timeline	Deliverables
Phase 1	Weeks 1-3	Core RAG pipeline (ingestion, embedding, retrieval, LLM response). Single-tab Streamlit UI. API key management.
Phase 2	Weeks 4-5	Voice assistant (STT/TTS), multi-tab UI (Query, Voice, Analytics, Settings). Dark theme CSS.
Phase 3	Weeks 6-7	Modular refactor into components/ / services/ / utils/ / config/ layers. Streaming LLM support. Analytics dashboard.
Phase 4 (Current)	Week 8	Final testing, documentation, capstone project report submission.

4. Project Design Description

The Document Intelligence Platform follows a layered, modular design philosophy. Each concern is isolated into a dedicated module, allowing independent testing, replacement, and scaling of components.

4.1 Module Design

[config/settings.py](#)

Centralizes all configurable parameters: API keys (loaded from .env via python-dotenv), LLM parameters (model name, temperature, max tokens), document processing parameters (chunk size, overlap, batch size, separators), retrieval parameters (K, search type), voice parameters (speech rate, timeout), and ElevenLabs voice ID mappings. This single-file configuration allows environment-specific overrides without code changes.

[services/llm_service.py](#)

The core RAG engine. The `get_llm()` function is decorated with `@st.cache_resource` to ensure the Groq LLM client is instantiated once per session. The `create_vector_embedding_from_files()` function orchestrates the full ingestion pipeline with user-facing progress reporting. The `process_query()` function builds the LCEL chain using the pipe operator (`|`) pattern: `retriever | prompt | llm | StrOutputParser()`.

[services/voice_service.py](#)

Manages all audio I/O. The ElevenLabs TTS function returns a base64-encoded MP3 wrapped in an HTML audio element with autoplay enabled, injected directly into the Streamlit UI via `st.markdown(html, unsafe_allow_html=True)`. The background listener uses a Python `threading.Thread` (`daemon=True`) and a `queue.Queue` for thread-safe audio transcription delivery.

[utils/document_loader.py](#)

Provides format-specific document loading with consistent LangChain Document output. The `load_docx_file()` function extracts text from both paragraphs and table cells, preserving structured document content. Temporary files are managed with Python's `tempfile` module and explicitly unlinked after loading.

[components/ \(UI Layer\)](#)

Each tab is a separate Python module rendering its content via a `render_*`() function called from `app_new.py`. This design enables tab-level unit testing and independent UI iteration. The `sidebar.py` component handles document upload and triggers the ingestion service. State transitions (e.g., vector database going online) are communicated via `st.session_state` and `st.rerun()`.

4.2 Technical Stack Summary

Table 7: Technical Stack Summary

Component	Technology	Purpose
Frontend	Streamlit	Interactive web interface with session state management
LLM Inference	Groq (Llama 3.1 8B)	~800 tokens/sec response generation via LPU hardware
Embeddings	OpenAI Ada-002	1536-dimensional semantic vector representations
Vector Database	FAISS (CPU)	In-memory similarity search over document chunks
RAG Orchestration	LangChain (LCEL)	Chain composition: retriever prompt LLM parser
Voice STT	Google Speech API	Microphone audio to text transcription
Voice TTS	ElevenLabs	Premium studio-quality text-to-speech synthesis
PDF Parsing	PyPDF (PyPDFLoader)	Extract text from multi-page PDF documents
DOCX Parsing	python-docx	Extract paragraphs and table cells from Word files
Config Management	python-dotenv	Runtime API key injection from .env file

5. Project Internal/External Interface Impacts and Specification

5.1 External API Interfaces

OpenAI Embeddings API

- Endpoint: <https://api.openai.com/v1/embeddings>
- Model: text-embedding-ada-002
- Input: Text string (up to 8,191 tokens per request)
- Output: 1536-dimensional float array
- Rate Limit Impact: Batch processing (BATCH_SIZE=10) limits concurrent requests. max_retries=3 handles transient 429 errors.
- Cost: ~\$0.0001 per 1,000 tokens
-

Groq LLM API

- Endpoint: <https://api.groq.com/openai/v1/chat/completions>
- Model: llama-3.1-8b-instant
- Input: System prompt + context + user question (up to 8,192 context tokens)
- Output: Completion text (up to 1,000 tokens)
- Throughput: ~800 tokens/sec on Groq LPU hardware
- Cost: Free tier available; no cost for standard academic use

ElevenLabs TTS API

- Endpoint: `https://api.elevenlabs.io/v1/text-to-speech/{voice_id}`
- Model: `eleven_turbo_v2_5`
- Input: Text string (up to 800 characters per request, enforced in code)
- Output: Audio/MPEG stream, base64-encoded for inline HTML playback
- Voices: Bella, Sarah, Harry, Liam, Jessica (configurable via `ELEVENLABS_VOICES` dict)

Google Speech Recognition

- Library: `SpeechRecognition` (`recognize_google` method)
- Input: Microphone audio captured via `pyaudio` (`SPEECH_TIMEOUT=10s`, `SPEECH_PHRASE_TIME_LIMIT=15s`)
- Output: Transcribed text string
- Fallback: Returns error string on `WaitTimeoutError`, `UnknownValueError`, or `RequestError`

5.2 Internal Component Interfaces

The primary internal interface is the Streamlit session state (`st.session_state`), which acts as the shared state bus between components:

- `st.session_state.vectors`: FAISS `VectorStore` instance shared between `llm_service.py` and all UI components.
- `st.session_state.final_documents`: List of LangChain Document chunks; used by `analytics_tab.py` for statistics.
- `st.session_state.chat_history`: List of dicts with keys `question`, `answer`, `timestamp`, `response_time`, `voice_query`.
- `st.session_state.last_spoken_text`: Stores most recent AI response for TTS replay.
- `st.session_state.audio_queue`: `queue.Queue` instance for thread-safe STT result delivery from background listener.

6. Project Design Units Impacts

6.1 Functional Area A: RAG Pipeline (`llm_service.py`)

6.1.1 Functional Overview

The RAG pipeline is the core intelligence layer of the platform. It is responsible for transforming raw document files into a queryable vector knowledge base and executing semantic retrieval-augmented generation queries. The pipeline is implemented in `services/llm_service.py` and uses LangChain's LCEL composition model for clean, maintainable chain construction.

6.1.2 Impacts

- Performance: Embedding generation time scales linearly with document size. A 100-page PDF produces ~400 chunks, requiring ~40 batch API calls (at `BATCH_SIZE=10`). Typical ingestion time: 30-90 seconds depending on document size and API latency.
- Memory: Each FAISS vector entry consumes 1536 floats (6KB). A 1,000-chunk corpus requires ~6MB of RAM, well within typical deployment constraints.
- Accuracy: The RAG approach achieves near-zero hallucination by constraining the LLM to retrieved context. Response quality depends on chunk size and retrieval K; current settings (1000 chars, K=4) were tuned for academic paper analysis.
- Scalability: The current FAISS flat L2 index has $O(n)$ search complexity. For corpora exceeding 100,000 chunks, FAISS IVF or HNSW indices should be considered.

6.1.3 Requirements

- REQ-RAG-01: The pipeline must complete vector embedding creation for a 50-page document within 120 seconds under normal API latency.
- REQ-RAG-02: The retriever must return exactly K=4 document chunks per query (configurable via RETRIEVAL_K).
- REQ-RAG-03: The prompt template must include explicit instructions preventing out-of-context responses.
- REQ-RAG-04: The LLM temperature must be set to 0.1 or lower for factual document analysis tasks.
-

6.2 Functional Area B: Voice Assistant (voice_service.py)

6.2.1 Functional Overview

The voice assistant module provides bidirectional audio interaction with the RAG system. It exposes two primary capabilities: speech-to-text conversion (allowing users to dictate queries) and text-to-speech synthesis (reading AI responses aloud). The module manages a background listener thread for non-blocking microphone capture and integrates ElevenLabs' premium voice synthesis for high-quality audio output.

6.2.2 Impacts

- User Experience: Voice interaction significantly improves accessibility, particularly for users who prefer hands-free operation or consume content while multitasking.
- Latency: ElevenLabs TTS adds ~0.5-1.5 seconds to the response cycle for audio synthesis. This is acceptable for conversational use.
- Browser Compatibility: Audio playback relies on the HTML5 <audio> element with autoplay. Some browsers (Safari, Firefox) may block autoplay; users are presented with a manual play button via the controls attribute.
- Thread Safety: The background listener uses queue. Queue for thread-safe STT result delivery, preventing race conditions with Streamlit's rerun mechanism.

6.2.3 Requirements

- REQ-VOICE-01: The STT module must handle microphone unavailability gracefully with a user-friendly error message, without crashing the application.
- REQ-VOICE-02: TTS audio must be delivered as an inline browser audio player (autoplay + controls) within 3 seconds of response generation.
- REQ-VOICE-03: Voice query results must be stored in chat_history with voice_query=True flag for analytics tracking.
- REQ-VOICE-04: The selected ElevenLabs voice must persist across the session via st.session_state.selected_voice.

7. Open Issues

Table 8: Open Issues

ID	Issue	Priority	Resolution Plan
OI-01	FAISS index is not persisted between sessions; users must re-upload and re-process documents on each session.	High	Implement FAISS.save_local() / FAISS.load_local() in v1.1 release.
OI-02	Scanned PDFs without text layers return empty content silently.	High	Add OCR preprocessing (pytesseract) and user warning for empty extraction in v1.1.
OI-03	Safari/Firefox may block HTML5 autoplay audio without user interaction.	Medium	Add explicit play button as primary control; autoplay as secondary enhancement.
OI-04	The analytics tab does not persist metrics beyond the current session.	Medium	Integrate lightweight SQLite session logging in v1.2.
OI-05	Large documents (>200 pages) may cause OpenAI embedding API rate limit errors.	Medium	Implement exponential backoff retry decorator on embedding calls.
OI-06	The settings tab UI does not apply changes to the active LLM instance mid-session.	Low	Add a 'Apply Settings' button that clears the @cache_resource cache and re-initializes LLM.

8. Acknowledgements

The author gratefully acknowledges the contributions of the following individuals, communities, and organizations that made this project possible:

- Governors State University, Department of Computer Science: For academic support and the graduate capstone seminar framework that guided this project's development.
- LangChain Community: The open-source LangChain framework and its LCEL (LangChain Expression Language) paradigm formed the backbone of the RAG pipeline implementation.
- Groq: For providing the ultra-fast LPU inference platform that enables sub-2-second LLM response times, critical for the interactive user experience.
- OpenAI: For the text-embedding-ada-002 model that powers the semantic vector search capabilities of the platform.
- ElevenLabs: For the premium text-to-speech API that delivers studio-quality voice synthesis in the voice assistant module.
- Meta AI Research (FAISS): For the open-source Facebook AI Similarity Search library that provides efficient in-memory vector similarity search.
- Streamlit Team: For the Python-native web application framework that enabled rapid UI development with minimal frontend overhead.

9. References

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.
- Johnson, J., Douze, M., & Jegou, H. (2019). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547.
- Chase, H. (2022). LangChain. GitHub Repository. <https://github.com/hwchase17/langchain>
- Streamlit Inc. (2023). Streamlit Documentation. <https://docs.streamlit.io>
- OpenAI. (2022). Text Embeddings. OpenAI API Documentation. <https://platform.openai.com/docs/guides/embeddings>
- Groq Inc. (2024). Groq API Documentation. <https://console.groq.com/docs>
- ElevenLabs. (2024). ElevenLabs API Documentation. <https://docs.elevenlabs.io>
- Wikipedia. (2024). SMART Criteria. https://en.wikipedia.org/wiki/SMART_criteria

10. Appendices

Appendix A: Project Directory Structure

```
Document Ai/
├── app_new.py           # New modular main application
├── requirements.txt     # Python dependencies
├── .env                # API keys (not in repo)
├── research papers/    # Default document directory
├── config/             # Configuration modules
│   ├── settings.py     # Application settings and API keys
│   ├── styling.py      # Dark theme CSS styling
│   └── prompts.py      # LLM prompt templates
├── utils/              # Utility functions
│   ├── document_loader.py # Document loading utilities
│   └── session_state.py  # Session state management
├── services/           # Core services
│   ├── llm_service.py  # LLM and vector processing (RAG)
│   └── voice_service.py # Voice assistant (TTS/STT)
└── components/         # UI components
    ├── sidebar.py      # Sidebar with system dashboard
    ├── query_tab.py    # Document query interface
    ├── voice_tab.py    # Voice assistant interface
    ├── analytics_tab.py # System analytics
    └── settings_tab.py  # Configuration settings
```

Appendix B: Environment Variables Reference

The following environment variables must be configured in the .env file before running the application:

9. **OPENAI_API_KEY** — OpenAI API key for text-embedding-ada-002 embeddings
10. **GROQ_API_KEY** — Groq API key for Llama 3.1 8B LLM inference
11. **ELEVEN_API_KEY** — ElevenLabs API key for premium text-to-speech synthesis

Appendix C: Key Configuration Parameters

12. **LLM_MODEL = "llama-3.1-8b-instant"**: Switch to "llama-3.1-70b-versatile" for higher quality at slower speed.
13. **LLM_TEMPERATURE = 0.1**: Keep low for factual document Q&A. Increase toward 0.7 for creative summarization tasks.
14. **CHUNK_SIZE = 1000 chars**: Larger chunks provide more context per retrieval; smaller chunks improve targeting precision.
15. **CHUNK_OVERLAP = 200 chars**: Prevents context loss at chunk boundaries. Recommended at 20% of chunk size.
16. **RETRIEVAL_K = 4**: Number of chunks retrieved per query. Increase for complex multi-part questions.
17. **BATCH_SIZE = 10**: Number of document chunks per OpenAI embedding API call. Reduce if rate limit errors occur.